

Keehan_Alyssa_HW1

Vertebral Column Data

Name: Alyssa Keehan **GitHub Username:** akeehan56

Email: akeehan@usc.edu

USC ID: 3429542433

(a) Download Data

Package imports

```
In [1]: # ! pip install scikit-learn
# ! pip install --upgrade scikit-learn
# ! pip install --upgrade pip
#! pip install --upgrade --force-reinstall --no-cache-dir \
#     numpy scipy scikit-learn pandas matplotlib seaborn
```

```
In [2]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
)
```

Get the Vertebral Column Data Set

```
In [3]: #initiate the column names at the beginning since they are not available in
column_names = [
    "pelvic_incidence",
    "pelvic_tilt",
    "lumbar_lordosis_angle",
    "sacral_slope",
    "pelvic_radius",
    "grade_spondylolisthesis",
    "class"
]

data = pd.read_csv(
    "../data/vertebral_column_data/column_2C.dat",
    sep=r"\s+",
```

```

    header=None,
    names=column_names
)

data["class"] = data["class"].map({"N0": 0, "AB": 1})

independent_variables = data.columns[:-1].tolist()

data.head()

```

Out[3]:

	pelvic_incidence	pelvic_tilt	lumbar_lordosis_angle	sacral_slope	pelvic_radius	gr
0	63.03	22.55	39.61	40.48	98.67	
1	39.06	10.06	25.02	29.00	114.41	
2	68.83	22.22	50.09	46.61	105.99	
3	69.30	24.65	44.31	44.64	101.87	
4	49.71	9.65	28.32	40.06	108.17	

(b) Pre-Processing and Exploratory Data Analysis

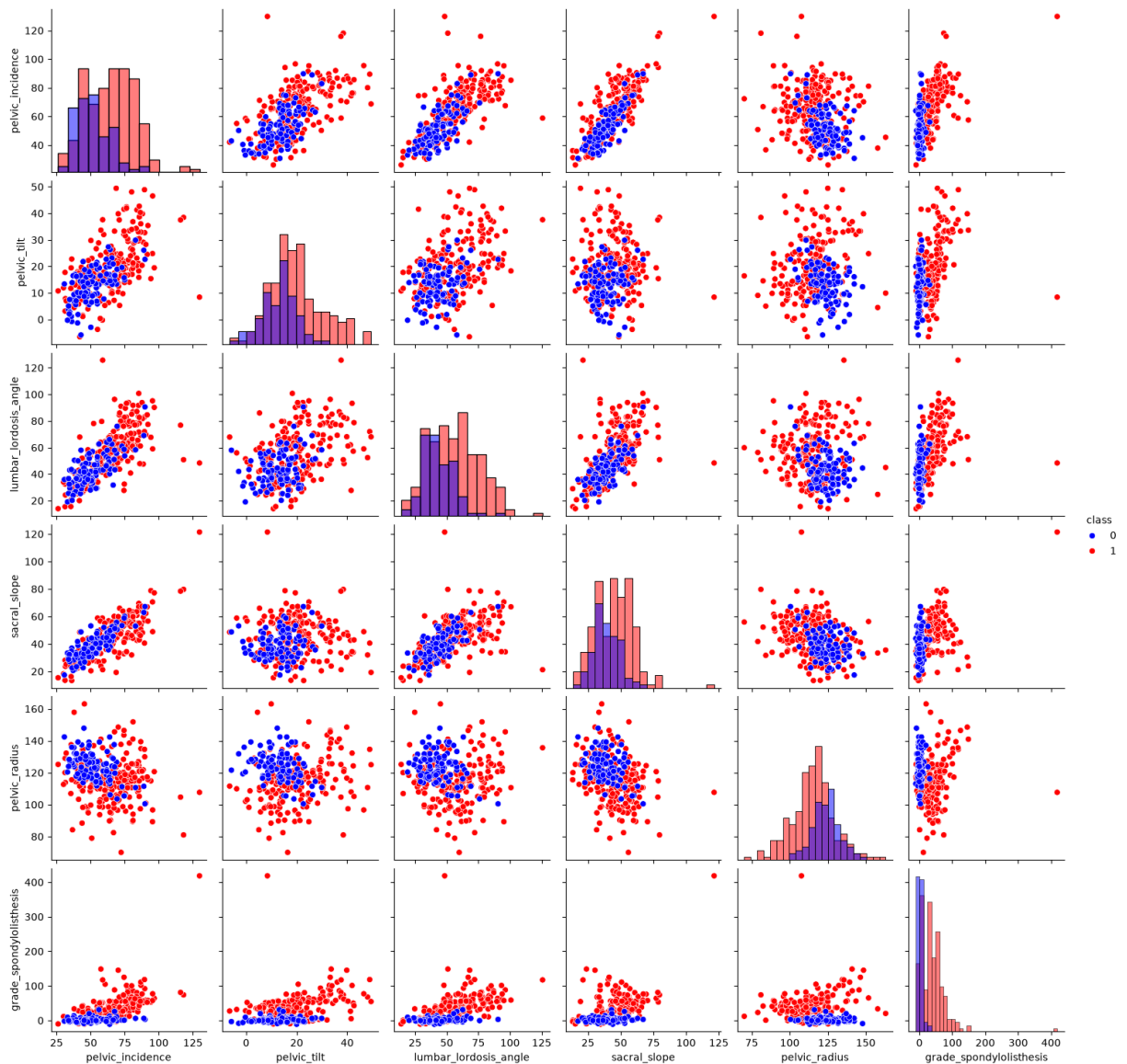
i. Scatterplots

```

In [4]: sns.pairplot(
    data,
    vars=independent_variables,
    hue="class",
    palette={0: "blue", 1: "red"},
    diag_kind="hist"
)

plt.show()

```



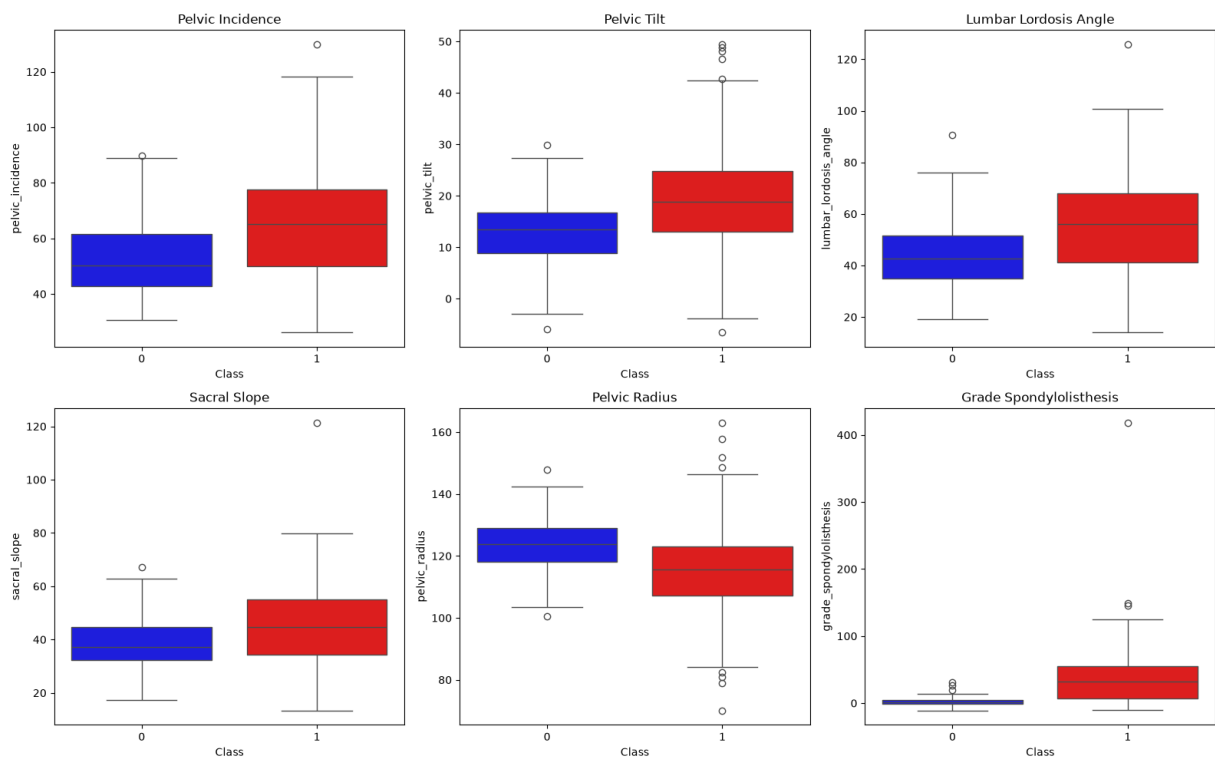
ii. Boxplots

```
In [5]: fig, axes = plt.subplots(2, 3, figsize=(16, 10))

for variable, ax in zip(independent_variables, axes.flatten()):
    sns.boxplot(
        data=data,
        x="class",
        y=variable,
        hue="class",
        palette={0: "blue", 1: "red"},
        legend=False,
        ax=ax
    )

    ax.set_title(variable.replace("_", " ").title())
    ax.set_xlabel("Class")

plt.tight_layout()
plt.show()
```



iii. Split Data Set

```
In [6]: class_0 = data[data["class"] == 0]
class_1 = data[data["class"] == 1]

training_set = pd.concat([
    class_0.iloc[:70],
    class_1.iloc[:140]
]).reset_index(drop=True)
print(training_set)

test_set = pd.concat([
    class_0.iloc[70:],
    class_1.iloc[140:]
]).reset_index(drop=True)
print(test_set)

X_train = training_set[independent_variables]
y_train = training_set["class"]

X_test = test_set[independent_variables]
y_test = test_set["class"]
```

	pelvic_incidence	pelvic_tilt	lumbar_lordosis_angle	sacral_slope	\
0	38.51	16.96	35.11	21.54	
1	54.92	18.97	51.60	35.95	
2	44.36	8.95	46.90	35.42	
3	48.32	17.45	48.00	30.87	
4	45.70	10.66	42.58	35.04	
..	...	...	...	...	
205	77.12	30.35	77.48	46.77	
206	88.02	39.84	81.77	48.18	
207	83.40	34.31	78.42	49.09	
208	72.05	24.70	79.87	47.35	
209	85.10	21.07	91.73	64.03	

	pelvic_radius	grade_spondylolisthesis	class
0	127.63	7.99	0
1	125.85	2.00	0
2	129.22	4.99	0
3	128.98	-0.91	0
4	130.18	-3.39	0
..	...	...	...
205	110.61	82.09	1
206	116.60	56.77	1
207	110.47	49.67	1
208	107.17	56.43	1
209	109.06	38.03	1

[210 rows x 7 columns]

	pelvic_incidence	pelvic_tilt	lumbar_lordosis_angle	sacral_slope	\
0	50.09	13.43	34.46	36.66	
1	64.26	14.50	43.90	49.76	
2	53.68	13.45	41.58	40.24	
3	49.00	13.11	51.87	35.88	
4	59.17	14.56	43.20	44.60	
..	...	...	...	...	
95	80.11	33.94	85.10	46.17	
96	95.48	46.55	59.00	48.93	
97	74.09	18.82	76.03	55.27	
98	87.68	20.37	93.82	67.31	
99	48.26	16.42	36.33	31.84	

	pelvic_radius	grade_spondylolisthesis	class
0	119.13	3.09	0
1	115.39	5.95	0
2	113.91	2.74	0
3	126.40	0.54	0
4	121.04	2.83	0
..	...	...	...
95	125.59	100.29	1
96	96.68	77.28	1
97	128.41	73.39	1
98	120.94	76.73	1
99	94.88	28.34	1

[100 rows x 7 columns]

(c) Classification

i. Euclidean Metric

```
In [7]: knn = KNeighborsClassifier(
        n_neighbors=4,
        metric="euclidean"
    )

    knn.fit(X_train, y_train)

    y_pred = knn.predict(X_test)
```

ii. Test Data

Test all the data in the test database with k nearest neighbors. Take decisions by majority pollin

```
In [8]: euclidean_k_values = list(range(208, 0, -3))

    euclidean_train_errors = []
    euclidean_test_errors = []

    for k in euclidean_k_values:
        knn = KNeighborsClassifier(
            n_neighbors=k,
            metric="euclidean"
        )

        knn.fit(X_train, y_train)

        train_predictions = knn.predict(X_train)
        test_predictions = knn.predict(X_test)

        euclidean_train_errors.append(
            1 - accuracy_score(y_train, train_predictions)
        )

        euclidean_test_errors.append(
            1 - accuracy_score(y_test, test_predictions)
        )

    best_euclidean_test_error = min(euclidean_test_errors)
    best_euclidean_train_error = min(euclidean_train_errors)

    best_euclidean_k = min(
        k for k, error in zip(euclidean_k_values, euclidean_test_errors)
        if error == best_euclidean_test_error
    )
```

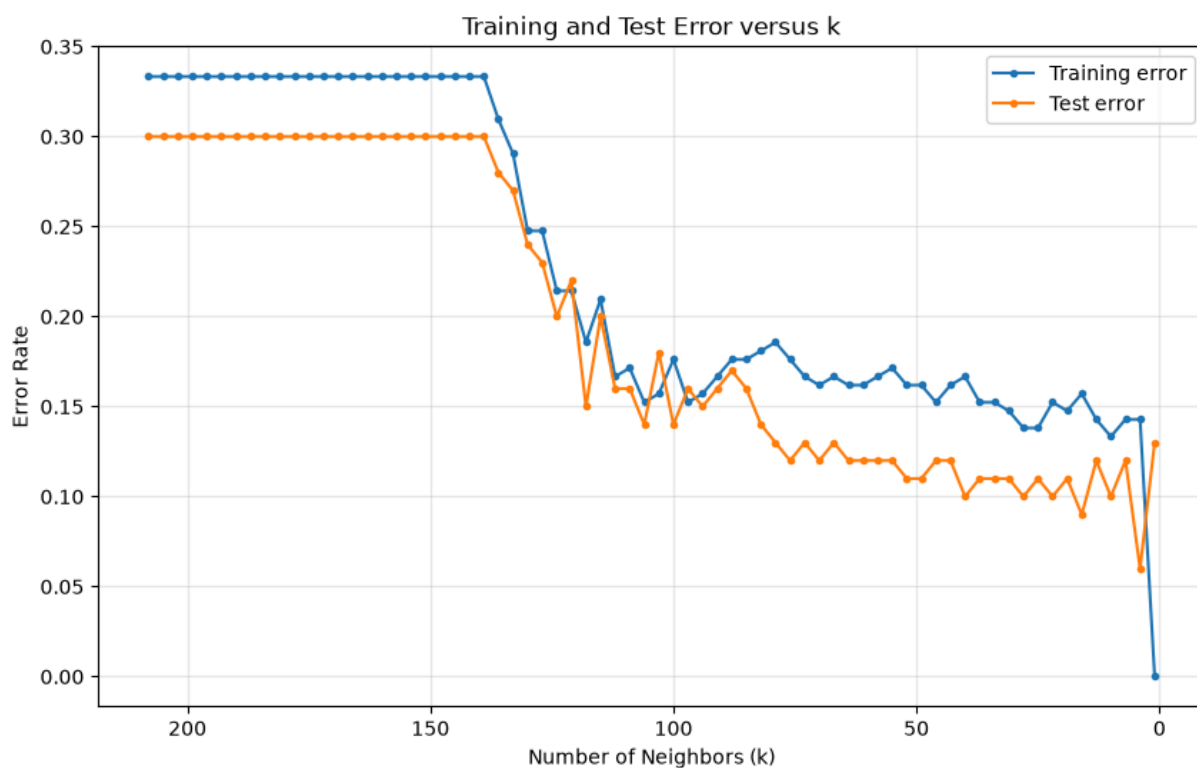
Plot train and test errors in terms of k for $k \in \{208, 205, \dots, 7, 4, 1\}$ (in reverse order)

```
In [9]: plt.figure(figsize=(10, 6))

plt.plot(
    euclidean_k_values,
    euclidean_train_errors,
    marker="o",
    markersize=3,
    label="Training error"
)

plt.plot(
    euclidean_k_values,
    euclidean_test_errors,
    marker="o",
    markersize=3,
    label="Test error"
)

plt.xlabel("Number of Neighbors (k)")
plt.ylabel("Error Rate")
plt.title("Training and Test Error versus k")
plt.gca().invert_xaxis()
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



Which k^* is the most suitable k among those values?

```
In [10]: error_results = pd.DataFrame({
    "k": euclidean_k_values,
    "train_error": euclidean_train_errors,
```

```

    "test_error": euclidean_test_errors
})

best_results = error_results[
    error_results["test_error"] == best_euclidean_test_error
]

k_star_euclidean = int(best_results["k"].min())

print("Best k:", k_star_euclidean)
print("Minimum test error:", best_euclidean_test_error)

best_results.sort_values("k")

```

Best k: 4

Minimum test error: 0.060000000000000005

Out[10]:

	k	train_error	test_error
68	4	0.142857	0.06

Display Confusion Matrix, true positive rate, true negative rate, precision, and F1-score when $k = k^*$

```

In [12]: best_knn = KNeighborsClassifier(
            n_neighbors=k_star_euclidean,
            metric="euclidean"
        )

best_knn.fit(X_train, y_train)
y_pred = best_knn.predict(X_test)

cm = confusion_matrix(y_test, y_pred)

tn, fp, fn, tp = cm.ravel()

true_positive_rate = tp / (tp + fn)
true_negative_rate = tn / (tn + fp)
precision = precision_score(y_test, y_pred, pos_label=1)
f1 = f1_score(y_test, y_pred, pos_label=1)

print(f"True positive rate: {true_positive_rate:.4f}")
print(f"True negative rate: {true_negative_rate:.4f}")
print(f"Precision: {precision:.4f}")
print(f"F1-score: {f1:.4f}")

```

True positive rate: 0.9857

True negative rate: 0.8333

Precision: 0.9324

F1-score: 0.9583

```

In [14]: plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",

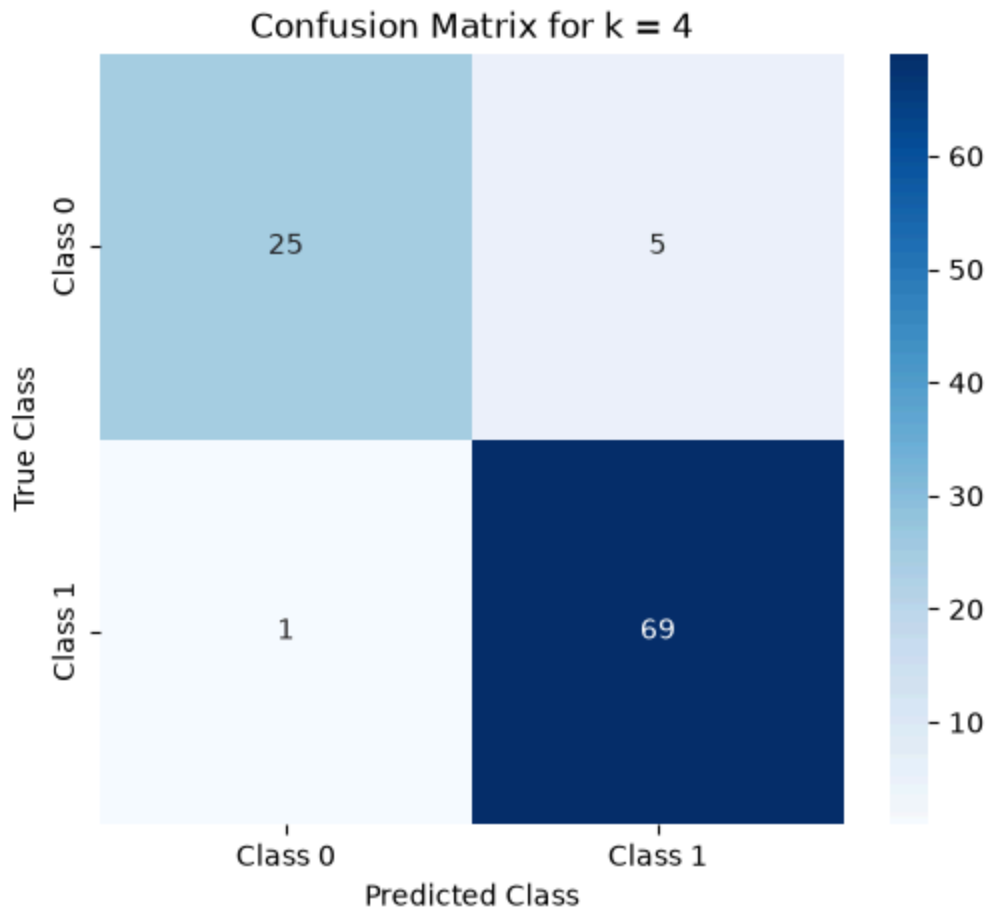
```

```

cmap="Blues",
xticklabels=["Class 0", "Class 1"],
yticklabels=["Class 0", "Class 1"]
)

plt.xlabel("Predicted Class")
plt.ylabel("True Class")
plt.title(f"Confusion Matrix for k = {k_star_euclidean}")
plt.show()

```



iii. Learning Curve

Re-run classifier on subsets of the data of various sizes from 10-210 in increments of 10

```

In [15]: N_values = list(range(10, 211, 10))

best_test_errors = []
best_k_values = []

training_class_0 = training_set[training_set["class"] == 0]
training_class_1 = training_set[training_set["class"] == 1]

for N in N_values:
    number_class_0 = N // 3
    number_class_1 = N - number_class_0

```

```

subset = pd.concat([
    training_class_0.iloc[:number_class_0],
    training_class_1.iloc[:number_class_1]
])

X_train_subset = subset[independent_variables]
y_train_subset = subset["class"]

# For N=200, this produces 1, 6, 11, ..., 196
candidate_k_values = list(range(1, N, 5))

errors_for_N = []

for k in candidate_k_values:
    knn = KNeighborsClassifier(
        n_neighbors=k,
        metric="euclidean"
    )

    knn.fit(X_train_subset, y_train_subset)
    predictions = knn.predict(X_test)

    test_error = 1 - accuracy_score(y_test, predictions)
    errors_for_N.append(test_error)

minimum_error = min(errors_for_N)

# Select the smallest k in case of a tie
optimal_k = min(
    k for k, error in zip(candidate_k_values, errors_for_N)
    if error == minimum_error
)

best_test_errors.append(minimum_error)
best_k_values.append(optimal_k)

```

```

In [16]: learning_curve_results = pd.DataFrame({
    "training_size": N_values,
    "optimal_k": best_k_values,
    "best_test_error": best_test_errors
})

learning_curve_results

```

Out [16]:

	training_size	optimal_k	best_test_error
0	10	1	0.25
1	20	6	0.20
2	30	1	0.22
3	40	11	0.25
4	50	26	0.30
5	60	21	0.29
6	70	26	0.29
7	80	31	0.29
8	90	41	0.29
9	100	6	0.25
10	110	6	0.22
11	120	16	0.17
12	130	16	0.16
13	140	16	0.15
14	150	16	0.13
15	160	6	0.13
16	170	6	0.13
17	180	6	0.10
18	190	6	0.09
19	200	6	0.09
20	210	6	0.08

Plot the Learning Curve

```

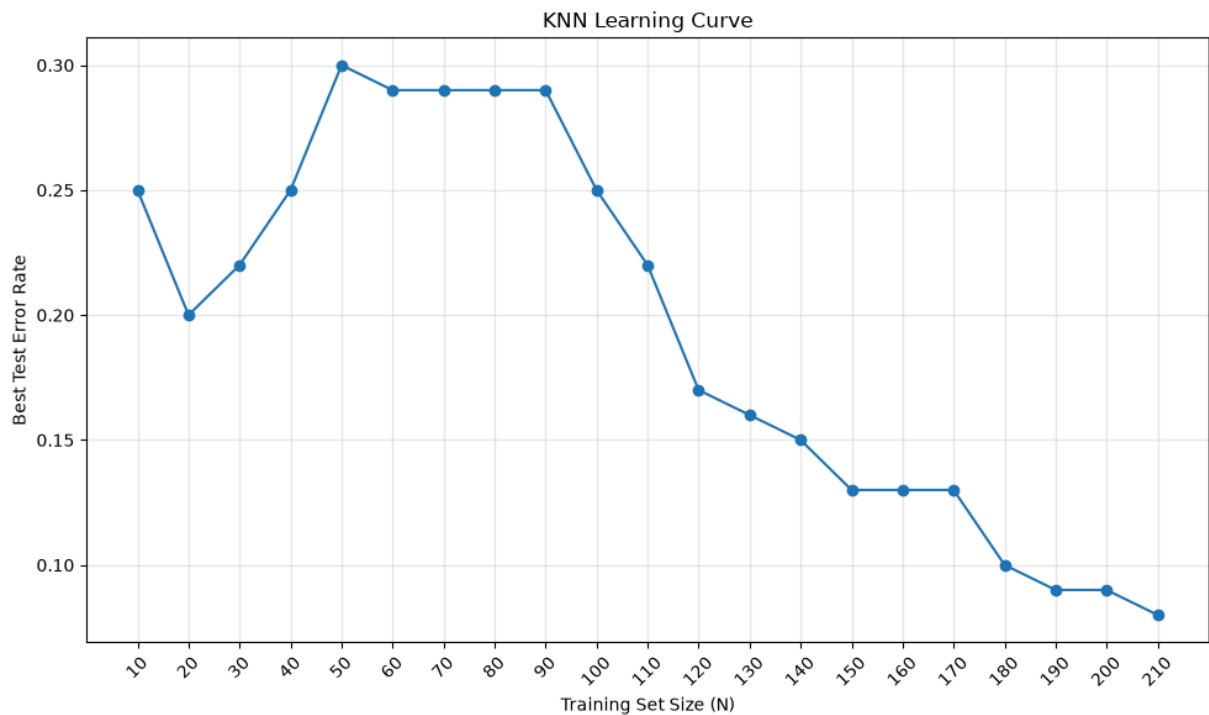
In [17]: plt.figure(figsize=(10, 6))

plt.plot(
    N_values,
    best_test_errors,
    marker="o"
)

plt.xlabel("Training Set Size (N)")
plt.ylabel("Best Test Error Rate")
plt.title("KNN Learning Curve")
plt.xticks(N_values, rotation=45)
plt.grid(alpha=0.3)

```

```
plt.tight_layout()
plt.show()
```



(d) Other Metrics

Replace the Euclidean metric with the following metrics and test them. Summarize the test errors (i.e., when $k=k^*$) in a table. Use all of your training data and select the best k when $\{1, 6, 11, \dots, 196\}$

```
In [18]: #initiate k values
k_values = list(range(1, 197, 5))
print(k_values)
```

```
[1, 6, 11, 16, 21, 26, 31, 36, 41, 46, 51, 56, 61, 66, 71, 76, 81, 86, 91, 96, 101, 106, 111, 116, 121, 126, 131, 136, 141, 146, 151, 156, 161, 166, 171, 176, 181, 186, 191, 196]
```

i. Minkowski Distance.

a. Manhattan Distance with $p = 1$.

```
In [19]: manhattan_train_errors = []
manhattan_test_errors = []

for k in k_values:
    knn = KNeighborsClassifier(
        n_neighbors=k,
        metric="manhattan"
    )

    knn.fit(X_train, y_train)
```

```
train_predictions = knn.predict(X_train)
test_predictions = knn.predict(X_test)

manhattan_train_errors.append(
    1 - accuracy_score(y_train, train_predictions)
)

manhattan_test_errors.append(
    1 - accuracy_score(y_test, test_predictions)
)

best_manhattan_test_error = min(manhattan_test_errors)
best_manhattan_train_error = min(manhattan_train_errors)

best_manhattan_k = min(
    k for k, error in zip(k_values, manhattan_test_errors)
    if error == best_manhattan_test_error
)

#store results in data frame
manhattan_results = pd.DataFrame({
    "k": k_values,
    "training_error": manhattan_train_errors,
    "test_error": manhattan_test_errors
})

manhattan_results
```

Out[19]:

	k	training_error	test_error
0	1	0.000000	0.12
1	6	0.138095	0.11
2	11	0.142857	0.11
3	16	0.133333	0.12
4	21	0.147619	0.12
5	26	0.166667	0.11
6	31	0.166667	0.12
7	36	0.161905	0.13
8	41	0.166667	0.13
9	46	0.171429	0.13
10	51	0.171429	0.14
11	56	0.185714	0.13
12	61	0.176190	0.15
13	66	0.180952	0.15
14	71	0.176190	0.14
15	76	0.180952	0.14
16	81	0.166667	0.16
17	86	0.152381	0.15
18	91	0.180952	0.20
19	96	0.176190	0.20
20	101	0.200000	0.22
21	106	0.204762	0.21
22	111	0.219048	0.23
23	116	0.228571	0.21
24	121	0.242857	0.22
25	126	0.261905	0.24
26	131	0.304762	0.28
27	136	0.323810	0.28
28	141	0.333333	0.30
29	146	0.333333	0.30
30	151	0.333333	0.30
31	156	0.333333	0.30

	k	training_error	test_error
32	161	0.333333	0.30
33	166	0.333333	0.30
34	171	0.333333	0.30
35	176	0.333333	0.30
36	181	0.333333	0.30
37	186	0.333333	0.30
38	191	0.333333	0.30
39	196	0.333333	0.30

```
In [20]: print("Best Manhattan k:", best_manhattan_k)
print("Best Manhattan test error:", best_manhattan_test_error)
```

Best Manhattan k: 6

Best Manhattan test error: 0.10999999999999999

b. With $\log_{10}(p)$ in $\{0.1, 0.2, 0.3, \dots, 1\}$.

```
In [21]: log10_p_values = np.arange(0.1, 1.01, 0.1)

minkowski_train_errors = []
minkowski_test_errors = []

for log10_p in log10_p_values:
    p = 10 ** log10_p

    knn = KNeighborsClassifier(
        n_neighbors=best_manhattan_k,
        metric="minkowski",
        p=p
    )

    knn.fit(X_train, y_train)

    train_predictions = knn.predict(X_train)
    test_predictions = knn.predict(X_test)

    minkowski_train_errors.append(
        1 - accuracy_score(y_train, train_predictions)
    )

    minkowski_test_errors.append(
        1 - accuracy_score(y_test, test_predictions)
    )

best_minkowski_test_error = min(minkowski_test_errors)
best_minkowski_train_error = min(minkowski_train_errors)

best_log10_p_minkowski = min(
```

```

log_p
for log_p, error in zip(log10_p_values, minkowski_test_errors)
    if error == best_minkowski_test_error
)

minkowski_results = pd.DataFrame({
    "log10(p)": np.round(log10_p_values, 1),
    "p": 10 ** log10_p_values,
    "train_error": minkowski_train_errors,
    "test_error": minkowski_test_errors
})

minkowski_results

```

Out[21]:

	log10(p)	p	train_error	test_error
0	0.1	1.258925	0.138095	0.09
1	0.2	1.584893	0.147619	0.09
2	0.3	1.995262	0.152381	0.08
3	0.4	2.511886	0.152381	0.08
4	0.5	3.162278	0.147619	0.08
5	0.6	3.981072	0.152381	0.06
6	0.7	5.011872	0.152381	0.07
7	0.8	6.309573	0.147619	0.08
8	0.9	7.943282	0.147619	0.09
9	1.0	10.000000	0.133333	0.09

```

In [22]: best_p = 10 ** best_log10_p_minkowski

print("Best log10(p):", round(best_log10_p_minkowski, 1))
print("Best p:", best_p)
print("Best Minkowski test error:", best_minkowski_test_error)

```

Best log10(p): 0.6

Best p: 3.981071705534973

Best Minkowski test error: 0.060000000000000005

C. Chebyshev Distance With $p \rightarrow \infty$.

```

In [23]: chebyshev_train_errors = []
chebyshev_test_errors = []

for k in k_values:
    knn = KNeighborsClassifier(
        n_neighbors=k,
        metric="chebyshev"
    )

    knn.fit(X_train, y_train)

```

```
train_predictions = knn.predict(X_train)
test_predictions = knn.predict(X_test)

chebyshev_train_errors.append(
    1 - accuracy_score(y_train, train_predictions)
)

chebyshev_test_errors.append(
    1 - accuracy_score(y_test, test_predictions)
)

best_chebyshev_test_error = min(chebyshev_test_errors)
best_chebyshev_train_error = min(chebyshev_train_errors)

best_chebyshev_k = min(
    k for k, error in zip(k_values, chebyshev_test_errors)
    if error == best_chebyshev_test_error
)

chebyshev_results = pd.DataFrame({
    "k": k_values,
    "train_error": chebyshev_train_errors,
    "test_error": chebyshev_test_errors
})

chebyshev_results
```

Out[23]:

	k	train_error	test_error
0	1	0.000000	0.13
1	6	0.152381	0.10
2	11	0.152381	0.12
3	16	0.152381	0.08
4	21	0.133333	0.11
5	26	0.157143	0.12
6	31	0.142857	0.12
7	36	0.152381	0.10
8	41	0.142857	0.13
9	46	0.161905	0.12
10	51	0.166667	0.12
11	56	0.166667	0.12
12	61	0.171429	0.14
13	66	0.161905	0.13
14	71	0.180952	0.14
15	76	0.176190	0.11
16	81	0.171429	0.14
17	86	0.171429	0.12
18	91	0.157143	0.15
19	96	0.161905	0.14
20	101	0.171429	0.19
21	106	0.176190	0.18
22	111	0.180952	0.18
23	116	0.176190	0.19
24	121	0.209524	0.21
25	126	0.223810	0.21
26	131	0.247619	0.25
27	136	0.319048	0.28
28	141	0.333333	0.30
29	146	0.333333	0.30
30	151	0.333333	0.30
31	156	0.333333	0.30

	k	train_error	test_error
32	161	0.333333	0.30
33	166	0.333333	0.30
34	171	0.333333	0.30
35	176	0.333333	0.30
36	181	0.333333	0.30
37	186	0.333333	0.30
38	191	0.333333	0.30
39	196	0.333333	0.30

```
In [24]: print("Best Chebyshev k:", best_chebyshev_k)
print("Best Chebyshev test error:", best_chebyshev_test_error)
```

Best Chebyshev k: 16

Best Chebyshev test error: 0.07999999999999996

ii. Mahalanobis Distance.

```
In [25]: X_train_array = X_train.to_numpy(dtype=float)
X_test_array = X_test.to_numpy(dtype=float)

covariance_matrix = np.cov(X_train_array, rowvar=False)
inverse_covariance_matrix = np.linalg.pinv(covariance_matrix)

mahalanobis_train_errors = []
mahalanobis_test_errors = []

for k in k_values:
    knn = KNeighborsClassifier(
        n_neighbors=k,
        metric="mahalanobis",
        metric_params={"VI": inverse_covariance_matrix},
        algorithm="brute"
    )

    knn.fit(X_train_array, y_train)

    train_predictions = knn.predict(X_train_array)
    test_predictions = knn.predict(X_test_array)

    mahalanobis_train_errors.append(
        1 - accuracy_score(y_train, train_predictions)
    )

    mahalanobis_test_errors.append(
        1 - accuracy_score(y_test, test_predictions)
    )

best_mahalanobis_test_error = min(mahalanobis_test_errors)
```

```
best_mahalanobis_train_error = min(mahalanobis_train_errors)

best_mahalanobis_k = min(
    k for k, error in zip(k_values, mahalanobis_test_errors)
    if error == best_mahalanobis_test_error
)

mahalanobis_results = pd.DataFrame({
    "k": k_values,
    "train_error": mahalanobis_train_errors,
    "test_error": mahalanobis_test_errors
})

mahalanobis_results
```

Out [25]:

	k	train_error	test_error
0	1	0.000000	0.17
1	6	0.119048	0.17
2	11	0.185714	0.20
3	16	0.176190	0.20
4	21	0.176190	0.21
5	26	0.185714	0.20
6	31	0.180952	0.23
7	36	0.180952	0.21
8	41	0.185714	0.24
9	46	0.185714	0.24
10	51	0.190476	0.24
11	56	0.190476	0.26
12	61	0.204762	0.25
13	66	0.195238	0.27
14	71	0.219048	0.26
15	76	0.214286	0.25
16	81	0.238095	0.24
17	86	0.233333	0.26
18	91	0.252381	0.26
19	96	0.242857	0.26
20	101	0.285714	0.28
21	106	0.300000	0.28
22	111	0.295238	0.31
23	116	0.323810	0.29
24	121	0.328571	0.30
25	126	0.328571	0.30
26	131	0.333333	0.30
27	136	0.333333	0.30
28	141	0.333333	0.30
29	146	0.333333	0.30
30	151	0.333333	0.30
31	156	0.333333	0.30

	k	train_error	test_error
32	161	0.333333	0.30
33	166	0.333333	0.30
34	171	0.333333	0.30
35	176	0.333333	0.30
36	181	0.333333	0.30
37	186	0.333333	0.30
38	191	0.333333	0.30
39	196	0.333333	0.30

```
In [26]: print("Best Mahalanobis k:", best_mahalanobis_k)
print("Best Mahalanobis test error:", best_mahalanobis_test_error)
```

Best Mahalanobis k: 1

Best Mahalanobis test error: 0.17000000000000004

```
In [27]: distance_summary_table = pd.DataFrame({

    "Distance Metric": [
        "Euclidean",
        "Manhattan",
        "Minkowski",
        "Chebyshev",
        "Mahalanobis"
    ],

    "Best k (or log(p) for Minkowski)": [
        k_star_euclidean,
        best_manhattan_k,
        best_log10_p_minkowski,
        best_chebyshev_k,
        best_mahalanobis_k
    ],

    "Best Test Error": [
        best_euclidean_test_error,
        best_manhattan_test_error,
        best_minkowski_test_error,
        best_chebyshev_test_error,
        best_mahalanobis_test_error
    ]

})

distance_summary_table["Best Test Error"] = distance_summary_table["Best Test Error"]
distance_summary_table
```

Out [27]:

	Distance Metric	Best k (or log(p) for Minkowski)	Best Test Error
0	Euclidean	4.0	0.06
1	Manhattan	6.0	0.11
2	Minkowski	0.6	0.06
3	Chebyshev	16.0	0.08
4	Mahalanobis	1.0	0.17

(e) Weighted Decision

Notes: purpose of weighting the distance is to prevent relatively distant neighbors from having the same influence as nearby observations, hence the inverse proportion detail in the instructions.

Weighted Euclidean Distance

```
In [28]: weighted_euclidean_train_errors = []
weighted_euclidean_test_errors = []

for k in k_values:
    knn = KNeighborsClassifier(
        n_neighbors=k,
        metric="euclidean",
        weights="distance"
    )

    knn.fit(X_train, y_train)

    train_predictions = knn.predict(X_train)
    test_predictions = knn.predict(X_test)

    weighted_euclidean_train_errors.append(
        1 - accuracy_score(y_train, train_predictions)
    )

    weighted_euclidean_test_errors.append(
        1 - accuracy_score(y_test, test_predictions)
    )

best_weighted_euclidean_test_error = min(weighted_euclidean_test_errors)
best_weighted_euclidean_train_error = min(weighted_euclidean_train_errors)

best_weighted_euclidean_k = min(
    k for k, error in zip(k_values, weighted_euclidean_test_errors)
    if error == best_weighted_euclidean_test_error
)

print("Best weighted Euclidean k:", best_weighted_euclidean_k)
print("Best weighted Euclidean test error:", best_weighted_euclidean_test_er
```

Best weighted Euclidean k: 6

Best weighted Euclidean test error: 0.09999999999999998

Weighted Manhattan Distance

```
In [29]: weighted_manhattan_train_errors = []
weighted_manhattan_test_errors = []

for k in k_values:
    knn = KNeighborsClassifier(
        n_neighbors=k,
        metric="manhattan",
        weights="distance"
    )

    knn.fit(X_train, y_train)

    train_predictions = knn.predict(X_train)
    test_predictions = knn.predict(X_test)

    weighted_manhattan_train_errors.append(
        1 - accuracy_score(y_train, train_predictions)
    )

    weighted_manhattan_test_errors.append(
        1 - accuracy_score(y_test, test_predictions)
    )

best_weighted_manhattan_test_error = min(weighted_manhattan_test_errors)
best_weighted_manhattan_train_error = min(weighted_manhattan_train_errors)

best_weighted_manhattan_k = min(
    k for k, error in zip(k_values, weighted_manhattan_test_errors)
    if error == best_weighted_manhattan_test_error
)

print("Best weighted Manhattan k:", best_weighted_manhattan_k)
print("Best weighted Manhattan test error:", best_weighted_manhattan_test_er
```

Best weighted Manhattan k: 26

Best weighted Manhattan test error: 0.09999999999999998

Weighted Chebyshev's Distance

```
In [30]: weighted_chebyshev_train_errors = []
weighted_chebyshev_test_errors = []

for k in k_values:
    knn = KNeighborsClassifier(
        n_neighbors=k,
        metric="chebyshev",
        weights="distance"
    )

    knn.fit(X_train, y_train)
```

```

train_predictions = knn.predict(X_train)
test_predictions = knn.predict(X_test)

weighted_chebyshev_train_errors.append(
    1 - accuracy_score(y_train, train_predictions)
)

weighted_chebyshev_test_errors.append(
    1 - accuracy_score(y_test, test_predictions)
)

best_weighted_chebyshev_test_error = min(weighted_chebyshev_test_errors)
best_weighted_chebyshev_train_error = min(weighted_chebyshev_train_errors)

best_weighted_chebyshev_k = min(
    k for k, error in zip(k_values, weighted_chebyshev_test_errors)
    if error == best_weighted_chebyshev_test_error
)

print("Best weighted Chebyshev k:", best_weighted_chebyshev_k)
print("Best weighted Chebyshev test error:", best_weighted_chebyshev_test_er

```

Best weighted Chebyshev k: 16

Best weighted Chebyshev test error: 0.10999999999999999

(f) Training Error Rate

```

In [31]: all_train_errors = pd.DataFrame({
    "Voting Method": [
        "Unweighted",
        "Unweighted",
        "Unweighted",
        "Unweighted",
        "Unweighted",
        "Unweighted",
        "Distance-weighted",
        "Distance-weighted",
        "Distance-weighted"
    ],
    "Distance Metric": [
        "Euclidean",
        "Manhattan",
        "Minkowski",
        "Chebyshev",
        "Mahalanobis",
        "Euclidean",
        "Manhattan",
        "Chebyshev"
    ],
    "Best Train Error": [
        best_euclidean_train_error,
        best_manhattan_train_error,
        best_minkowski_train_error,

```

```

        best_chebyshev_train_error,
        best_mahalanobis_train_error,
        best_weighted_euclidean_train_error,
        best_weighted_manhattan_train_error,
        best_weighted_chebyshev_train_error
    ]
})

all_train_errors["Best Train Error"] = all_train_errors["Best Train Error"].
all_train_errors

```

Out [31]:

	Voting Method	Distance Metric	Best Train Error
0	Unweighted	Euclidean	0.0000
1	Unweighted	Manhattan	0.0000
2	Unweighted	Minkowski	0.1333
3	Unweighted	Chebyshev	0.0000
4	Unweighted	Mahalanobis	0.0000
5	Distance-weighted	Euclidean	0.0000
6	Distance-weighted	Manhattan	0.0000
7	Distance-weighted	Chebyshev	0.0000

The lowest training error achieved in this assignment was 0.00, indicating perfect classification of the training data. This occurred because most methods included (k=1), meaning that each training observation was its own nearest neighbor and was therefore classified correctly. The only method with a nonzero minimum training error was unweighted Minkowski distance. This method used the optimal (k) found for Manhattan distance, which was (k*=6), rather than (k=1). Because each prediction used multiple neighbors, some training error was expected.

(f) Testing Error (Summary)

```

In [32]: all_test_errors = pd.DataFrame({
    "Voting Method": [
        "Unweighted",
        "Unweighted",
        "Unweighted",
        "Unweighted",
        "Unweighted",
        "Distance-weighted",
        "Distance-weighted",
        "Distance-weighted"
    ],
    "Distance Metric": [
        "Euclidean",

```

```

        "Manhattan",
        "Minkowski",
        "Chebyshev",
        "Mahalanobis",
        "Euclidean",
        "Manhattan",
        "Chebyshev"
    ],

    "Best Test Error": [
        best_euclidean_test_error,
        best_manhattan_test_error,
        best_minkowski_test_error,
        best_chebyshev_test_error,
        best_mahalanobis_test_error,
        best_weighted_euclidean_test_error,
        best_weighted_manhattan_test_error,
        best_weighted_chebyshev_test_error
    ]
})

all_test_errors["Best Test Error"] = all_test_errors["Best Test Error"].round(2)
all_test_errors

```

Out[32]:

	Voting Method	Distance Metric	Best Test Error
0	Unweighted	Euclidean	0.06
1	Unweighted	Manhattan	0.11
2	Unweighted	Minkowski	0.06
3	Unweighted	Chebyshev	0.08
4	Unweighted	Mahalanobis	0.17
5	Distance-weighted	Euclidean	0.10
6	Distance-weighted	Manhattan	0.10
7	Distance-weighted	Chebyshev	0.11

Answer

The lowest test error achieved in this assignment was 0.06, with a tie between unweighted Euclidean and unweighted Minkowski distances. When distance-weighted voting was used, the test error increased. This suggests that, for this dataset and train-test split, giving closer neighbors greater voting influence did not improve classification. Uniform voting generalized better because treating all selected neighbors equally produced more accurate test predictions.

In summary, although most methods achieved a training error of 0.00, this did not necessarily make them the best classifiers for unseen data. In fact, unweighted Minkowski KNN had the highest minimum training error but tied for the lowest test error.

This demonstrates the important distinction between the two measures: training error reflects how well a model fits the data on which it was trained, whereas test error reflects how well the model generalizes to unseen data.